

UNIVERSIDAD RAFAEL LANDÍVAR

ÁREA DE INGENIERÍA

Programación Web



REPASO

Ejercicio de repaso con MySQL y Node.js

AUTOR

Sajvin Gómez, Mario Daniel (1612921)

DOCENTE

Inge. Ricardo Trujillo

Código

```
const express = require("express");
const pool = require("../db");

const app = express();
const port = 3000;

app.use(express.json());

// endpoint para registrar un nuevo cliente
app.post("/clientes", (req, res) => {
  const { nombre, telefono, email } = req.body;

  // consulta parametrizada para insertar un nuevo usuarios
  const sqlQuery =
    "INSERT INTO clientes (nombre, telefono, email) VALUES (?, ?, ?)";

  // usar el pool de conexiones para ejecutar la consulta
  pool.query(sqlQuery, [nombre, telefono, email], (err, results) => {
    if (err) {
      console.log("Error al ejecutar la consulta: ", err);
      return res.status(500).send("Error de consulta");
    }
    res.status(201).json({
      message: "Cliente agregado con exito",
      usuarioId: results.insertId,
    });
  });
});

// endpoint para registrar una nueva mesa
app.post("/mesas", (req, res) => {
  const { numero, capacidad } = req.body;

  // consulta parametrizada para insertar un nuevo usuarios
  const sqlQuery = "INSERT INTO mesas (numero, capacidad) VALUES (?, ?)";

  // usar el pool de conexiones para ejecutar la consulta
  pool.query(sqlQuery, [numero, capacidad], (err, results) => {
    if (err) {
      console.log("Error al ejecutar la consulta: ", err);
      return res.status(500).send("Error de consulta");
    }
    res.status(201).json({
```

```

        message: "Mesa agregada con éxito",
        usuarioId: results.insertId,
    });
});
});

//endpoint para crear una reserva
app.post("/reservas", (req, res) => {
    const { cliente_id, mesa_id, fecha, hora } = req.body;

    // Verificar si la mesa está disponible
    const checkQuery =
        "SELECT * FROM reservas WHERE mesa_id = ? AND fecha = ? AND hora = ?";
    pool.query(checkQuery, [mesa_id, fecha, hora], (err, results) => {
        if (err) {
            console.error("Error al verificar disponibilidad: ", err);
            return res.status(500).send("Error en la consulta");
        }
        if (results.length > 0) {
            return res
                .status(400)
                .json({ message: "La mesa ya está reservada en esa fecha y hora" });
        }

        // Insertar la nueva reserva
        const insertQuery =
            "INSERT INTO reservas (cliente_id, mesa_id, fecha, hora) VALUES (?, ?, ?, ?)";
        pool.query(
            insertQuery,
            [cliente_id, mesa_id, fecha, hora],
            (err, results) => {
                if (err) {
                    console.error("Error al crear la reserva: ", err);
                    return res.status(500).send("Error en la consulta");
                }
                res.status(201).json({
                    message: "Reserva creada con éxito",
                    reservaId: results.insertId,
                });
            }
        );
    });
});
});
});

```

```

// endpoint para obtener todas las reservas con datos del cliente y la mesa
app.get("/reservas", (req, res) => {
  const sqlQuery = `
    SELECT reservas.id, clientes.nombre AS cliente, clientes.telefono,
    clientes.email,
    mesas.numero AS mesa, mesas.capacidad, reservas.fecha,
    reservas.hora, reservas.estado
    FROM reservas
    JOIN clientes ON reservas.cliente_id = clientes.id
    JOIN mesas ON reservas.mesa_id = mesas.id`;

  pool.query(sqlQuery, (err, results) => {
    if (err) {
      console.error("Error al obtener las reservas: ", err);
      return res.status(500).send("Error en la consulta");
    }
    res.json(results);
  });
});

// endpoint para actualizar el estado de una reserva
app.put("/reservas/:id", (req, res) => {
  const { id } = req.params;
  const { estado } = req.body;

  const updateQuery = "UPDATE reservas SET estado = ? WHERE id = ?";
  pool.query(updateQuery, [estado, id], (err, results) => {
    if (err) {
      console.error("Error al actualizar la reserva: ", err);
      return res.status(500).send("Error en la consulta");
    }
    if (results.affectedRows === 0) {
      return res.status(404).json({ message: "Reserva no encontrada" });
    }
    res.json({ message: "Estado de la reserva actualizado" });
  });
});

// endpoint para cancelar una reserva
app.delete("/reservas/:id", (req, res) => {
  const { id } = req.params;

  const deleteQuery = "DELETE FROM reservas WHERE id = ?";
  pool.query(deleteQuery, [id], (err, results) => {
    if (err) {

```

```

        console.error("Error al cancelar la reserva: ", err);
        return res.status(500).send("Error en la consulta");
    }
    if (results.affectedRows === 0) {
        return res.status(404).json({ message: "Reserva no encontrada" });
    }
    res.json({ message: "Reserva cancelada con éxito" });
});
});

// endpoint para consultar disponibilidad de mesas en una fecha y hora
específica
app.get("/reservas/disponibilidad", (req, res) => {
    const { fecha, hora } = req.query;

    const disponibilidadQuery = `
        SELECT * FROM mesas WHERE id NOT IN (
            SELECT mesa_id FROM reservas WHERE fecha = ? AND hora = ?
        )`;

    pool.query(disponibilidadQuery, [fecha, hora], (err, results) => {
        if (err) {
            console.error("Error al consultar disponibilidad: ", err);
            return res.status(500).send("Error en la consulta");
        }
        res.json({ disponibles: results });
    });
});

app.listen(port, () => {
    console.log(`Servidor corriendo en http://localhost:${port}`);
});

```

Registrar un nuevo cliente

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/clientes
- Body (JSON):**

```
1 {
2   "nombre": "Allan",
3   "telefono": 87456123,
4   "email": "allan@gamil.com"
5 }
```
- Status:** 201 Created
- Size:** 54 Bytes
- Time:** 15 ms
- Response (JSON):**

```
1 {
2   "message": "Cliente agregado con exito",
3   "usuarioId": 3
4 }
```

Registrar una nueva mesa

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/mesas
- Body (JSON Content):**

```
1 {
2   "numero": 1,
3   "capacidad": 6
4 }
```
- Status:** 201 Created
- Size:** 51 Bytes
- Time:** 88 ms
- Response (JSON):**

```
1 {
2   "message": "Mesa agregada con exito",
3   "usuarioId": 1
4 }
```

Crear una reserva, validando que la mesa este disponible

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/reservas
- Status:** 201 Created
- Size:** 53 Bytes
- Time:** 94 ms
- Body (JSON):**

```
1 {
2   "cliente_id": 1,
3   "mesa_id": 1,
4   "fecha": "2025-04-14",
5   "hora": "18:00:00"
6 }
```
- Response (JSON):**

```
1 {
2   "message": "Reserva creada con éxito",
3   "reservaId": 1
4 }
```

Obtener todas las reservas con datos del cliente y de la mesa

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:3000/reservas
- Status:** 200 OK
- Size:** 175 Bytes
- Time:** 70 ms
- Body (JSON):**

```
1 [
2   {
3     "id": 1,
4     "cliente": "Daniel",
5     "telefono": "12345678",
6     "email": "daniel@gamil.com",
7     "mesa": 1,
8     "capacidad": 6,
9     "fecha": "2025-04-14T06:00:00.000Z",
10    "hora": "18:00:00",
11    "estado": "pendiente"
12  }
13 ]
```

Actualizar el estado de una reserva

JS index.js JS db.js TC localhost:3000/productos X

PUT http://localhost:3000/reservas/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "estado": "confirmada"
3 }
```

Status: 200 OK Size: 46 Bytes Time: 90 ms

Response Headers 6 Cookies Results Docs {}

```
1 {
2   "message": "Estado de la reserva actualizado"
3 }
```

Result Grid Filter Rows: Edit:

	id	cliente_id	mesa_id	fecha	hora	estado
▶	1	1	1	2025-04-14	18:00:00	pendiente
•	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid Filter Rows: Edit:

	id	cliente_id	mesa_id	fecha	hora	estado
▶	1	1	1	2025-04-14	18:00:00	confirmada
•	NULL	NULL	NULL	NULL	NULL	NULL

Cancelar una reserva

JS index.js JS db.js TC localhost:3000/productos X

DELETE http://localhost:3000/reservas/1 Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

Status: 200 OK Size: 42 Bytes Time: 81 ms

Response Headers 6 Cookies Results Docs {}

```
1 {
2   "message": "Reserva cancelada con éxito"
3 }
```


Result Grid

Filter Rows:

Edit:

	id	cliente_id	mesa_id	fecha	hora	estado
*	NULL	NULL	NULL	NULL	NULL	NULL

Consultar disponibilidad de mesas en una fecha y hora específica

JS index.js JS db.js TC localhost:3000/productos X

GET http://localhost:3000/reservas/disponibilidad?fecha=2025-03-20 Send

Status: 200 OK Size: 51 Bytes Time: 75 ms

Query Headers 2 Auth Body 1 Tests Pre Run

Response Headers 6 Cookies Results Docs {}

Query Parameters

- ☒ fecha 2025-03-20
- ☒ hora 18:00:00
- ☐ parameter value

```
1 {
2   "disponibles": [
3     {
4       "id": 1,
5       "numero": 1,
6       "capacidad": 6
7     }
8   ]
9 }
```